

Fuzzy Private Set Union avec Chiffrement Homomorphe de Classe $\mathcal{CL}$

Intégration de BICYCL dans volePSI – Comparaison et Évaluation

Yasser NAMATY

Laboratoire Jean Kuntzmann

Encadrants : Aude Maignan / Jean-Guillaume Dumas

2026

Plan de la présentation

- 1 Contexte et objectifs
- 2 De la Private Set Union à la Fuzzy PSU
- 3 Résultats expérimentaux
- 4 Comparaison BICYCL vs Paillier
- 5 Perspectives : PIR et SimplePIR
- 6 Conclusion

Cadre

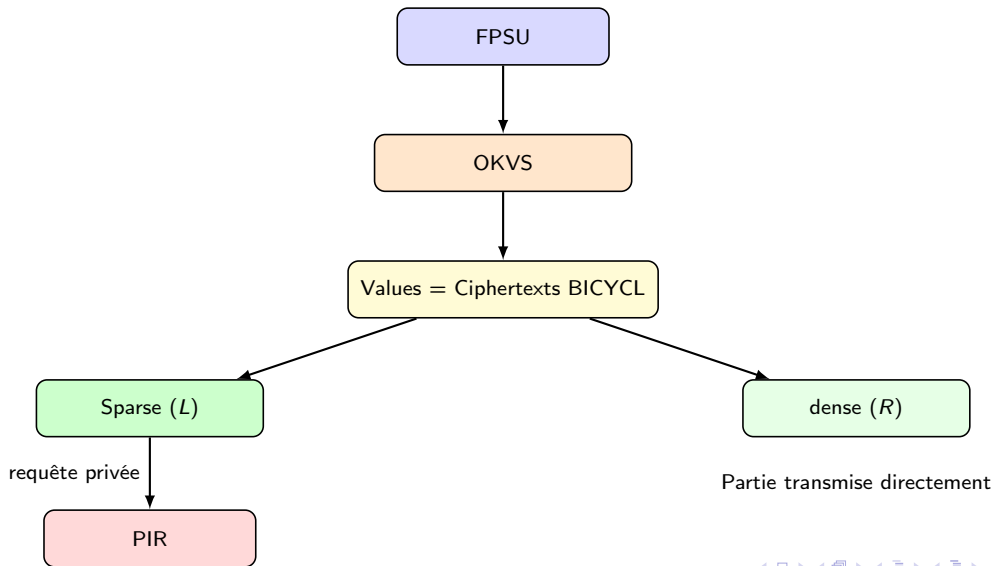
- Projet mené au **Laboratoire Jean Kuntzmann (LJK)**
- Encadrement : Aude Maignan, Jean-Guillaume Dumas
- Domaine : cryptographie appliquée, calcul multipartite sécurisé (MPC)

Objectif

Réaliser une **Fuzzy Private Set Union (FPSU)** en combinant :

- une **OKVS** (*Oblivious Key-Value Store*) ;
- un schéma de **chiffrement homomorphe linéaire (LHE)** (*Linearly Homomorphic Encryption*).

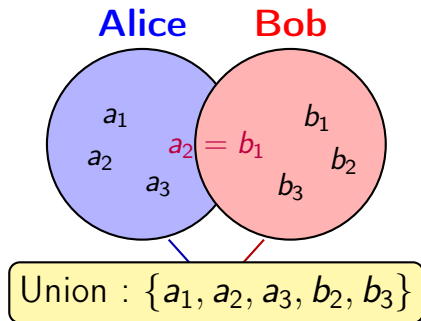
Architecture de l'implémentation



Private Set Union (PSU) : définition et principe

Problème

Deux parties, chacune détenant un ensemble d'éléments privés, souhaitent calculer l'union de leurs ensembles sans révéler d'informations supplémentaires.



- Applications : données médicales, détection de menaces, publicité ciblée
- **Objectif** : calculer l'union sans révéler les éléments individuels
- Ici, a_2 et b_1 sont fusionnés car **strictement égaux**

De la PSU à la Fuzzy PSU

Limite de la PSU

Fusion uniquement des éléments **identiques**.

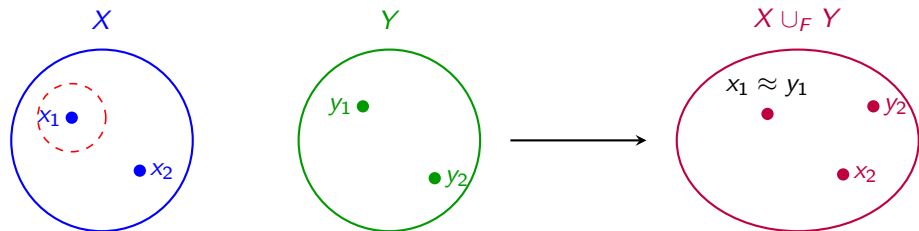
Pourquoi ?

Les données réelles sont souvent **bruitées** : biométrie, GPS, ADN, texte.

Fuzzy PSU

Remplacer l'égalité stricte par une **comparaison par distance** avec un seuil δ .

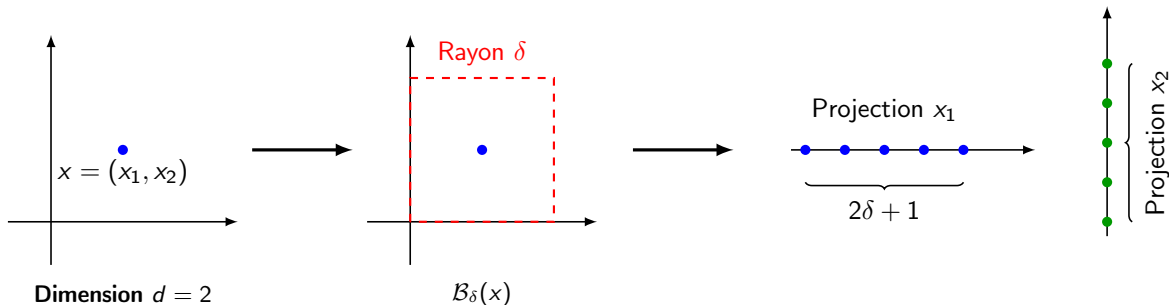
Fuzzy Private Set Union



Cercle pointillé : seuil de tolérance δ

y_1 est suffisamment proche de x_1 : un seul élément est conservé.

Dimension, rayon et projections



Principe

- Chaque élément est un point de dimension d .
- Le rayon δ définit la tolérance (**boule**).
- La boule est ensuite projetée sur chaque dimension.
- Dans notre implémentation ($d = 2$), une projection génère $2\delta + 1$ clés (offset= $-\delta \dots + \delta$).

Pourquoi un OKVS ?

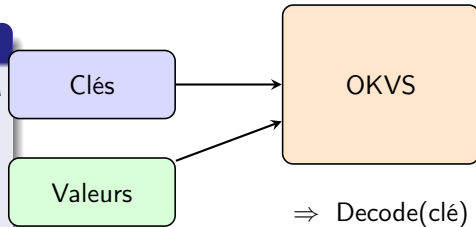
Principe

Dans une FPSU, on évite de comparer chaque élément de A avec chaque élément de B .

Chaque projection est donc encodée sous forme de paire

(clé, valeur)

puis stockée dans un **OKVS**.



But

Retrouver rapidement la valeur associée à une clé, sans révéler les clés utilisées.

Que contient l'OKVS ?

Clés

Les projections générées autour de chaque élément, soit $2\delta + 1$ projections par dimension.

Valeurs

À chaque clé est associée une part aléatoire :

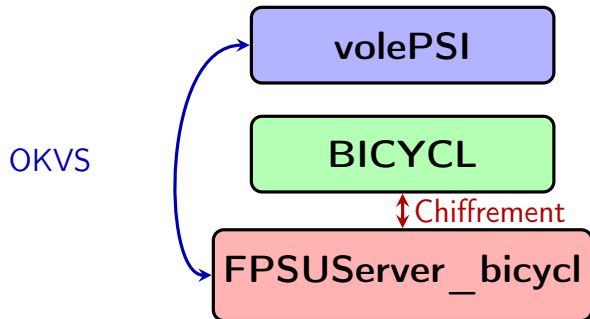
$$r_{i1}, r_{i2}, \dots, r_{id}$$

avec

$$\sum_{k=1}^d r_{ik} = 0$$

Ainsi :

- si les deux parties décodent la même clé, leurs parts s'annulent ;
- sinon, elles restent aléatoires.



- OKVS (Paxos)
- Corrélations VOLE
- $\mathcal{CL}$ -HSMqk

Pourquoi un chiffrement homomorphe ?

Après l'OKVS

Chaque projection est associée à une **valeur secrète** (encodée dans l'OKVS).

Pour tester une correspondance, il faut vérifier

$$r_{i1} + r_{i2} = 0$$

sans révéler r_{i1} ni r_{i2} .

Solution

On chiffre les valeurs avant leur stockage dans l'OKVS.
Grâce à l'homomorphie additive :

$$E(r_{i1}) \oplus E(r_{i2}) = E(r_{i1} + r_{i2})$$

Une seule opération de déchiffrement est effectuée à la fin.

Pourquoi BICYCL ?

- Chiffrement **homomorphe additif**
- Compatible avec le protocole FPSU
- Bibliothèque C++ performante et intégrable dans volePSI

Comment fonctionne BICYCL ? – Chiffrer et déchiffrer

Deux sous-groupes emboîtés

- Un sous-groupe d'ordre **connu** et premier q : sert à **encoder le message** m (via un générateur f de ce sous-groupe)
- Le reste du groupe, d'ordre **inconnu** : sert à **masquer** le message (aléa de chiffrement)

Vue d'ensemble du schéma

- 1 **KeyGen** : tire une clé secrète x , publie $pk = g^x$ dans le groupe de classes
- 2 **Encrypt**(m) : tire un aléa r , calcule $c_1 = g^r$ et $c_2 = pk^r \cdot f^m$ (composition de formes, via NUCOMP)
- 3 **Decrypt** : retire le masque avec x , puis retrouve m car il vit dans le **petit** sous-groupe d'ordre q (calcul de logarithme discret facile car q est connu et petit)
- 4 **Add** : composer directement les formes des deux chiffrés $\Rightarrow$ chiffré de la somme

Test avec $n = 8$, $m = 5$

- Indices 0 et 1 : clés synchronisées entre les deux serveurs → **décodage réussi**
- Indice 2 : clés désynchronisées → **échec attendu**
- Indices 3 et 4 : clés aléatoires → **échec attendu**

Interprétation

- Le décodage de l'OKVS et la reconstruction homomorphe fonctionnent comme prévu
- Les éléments réellement présents dans les deux ensembles sont correctement retrouvés
- Les éléments absents sont correctement rejetés, sans fausse acceptation observée

Critère	BICYCL	Paillier	Vainqueur
Taille des chiffrés	4 663 bits	6 144 bits	BICYCL
Génération de clés	24.45 ms	183–1353 ms	BICYCL
Chiffrement	9.44 ms	85–164 ms	BICYCL
Déchiffrement	23.58 ms	3.7–22 ms	Paillier
Addition homomorphe	11 073 μ s	70 μ s	Paillier

Conclusion du benchmark

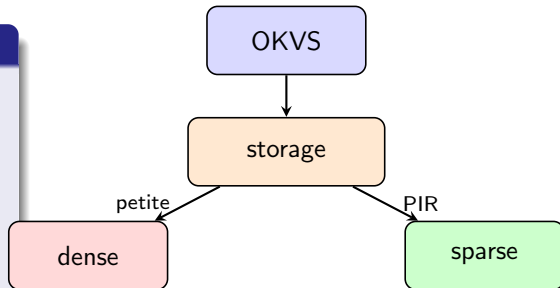
BICYCL est préférable quand la **compacité** et le **coût de chiffrement** dominant (communication, nombreux clients) ; Paillier reste préférable quand le protocole effectue **beaucoup d'additions homomorphes** par exécution.

Découpage

L'OKVS se décompose en :

- **storage** : taille $\approx n + \log n$,
- **dense** : taille $\approx \log n$,
- **sparse** : taille $\approx n$.

La partie **sparse** est la plus grande et celle qu'on traite avec SimplePIR.



On cherche seulement les **3 bits** correspondant au hachage.

Objectif

Implémenter les variantes suivantes :

- **SparseDecode1**

- sans PIR ;
- avec SimplePIR.

- **SparseDecode32**

- sans PIR ;
- avec SimplePIR.

Intégration de SimplePIR

Le **Private Information Retrieval (PIR)** sera intégré afin de **minimiser l'accès au vecteur encodé (storage)** de l'OKVS.

Au lieu de parcourir ou de transférer l'ensemble du storage, seules les quelques positions nécessaires au **SparseDecode** seront récupérées de manière privée.

Contributions

- ① Intégration de BICYCL dans volePSI pour le protocole FPSU
- ② Campagne de benchmarks BICYCL vs Paillier à niveau de sécurité égal
- ③ Analyse du rôle de chaque brique (OKVS, chiffrement homomorphe, PIR) dans la *Fuzzy* PSU

Résultats clés

- BICYCL : chiffrés 24% plus compacts, chiffrement jusqu'à $17\times$ plus rapide
- Coût : addition homomorphe environ $158\times$ plus lente que Paillier
- **Compromis favorable** pour les applications où la compacité et le chiffrement dominant

Limites actuelles

- Tests de validation menés sur de petits ensembles ($n = 8$)
- SimplePIR encore non relié à FPSU

Améliorations possibles

- 1 Chiffrement/déchiffrement par lots (*batch*)
- 2 Passage à l'échelle sur des ensembles de taille réaliste
- 3 Intégration complète du bridge PIR

Je remercie chaleureusement

Aude Maignan et **Jean-Guillaume Dumas**

pour leur encadrement tout au long de ce projet,

ainsi que l'ensemble du **Laboratoire Jean Kuntzmann**
pour son accueil et les échanges scientifiques.

Merci pour votre attention

Merci pour votre attention !

Questions ?